OGC® DOCUMENT: 23-003

External identifier of this OGC® document: http://www.opengis.net/doc/IS/ogcapi-connectedsystems-3/1.0

Open Geospatial Consortium

OGC API - CONNECTED SYSTEMS - PART 3: PUBLISH/SUBSCRIBE

STANDARD
Implementation

CANDIDATE SWG DRAFT

Version: 1.0
Submission Date: XXX
Approval Date: XXX
Publication Date:
Editor: Ian Patterson

Open Geospatial Consortium

License Agreement

Use of this document is subject to the license agreement at https://www.ogc.org/license

Suggested additions, changes and comments on this document are welcome and encouraged. Such suggestions may be submitted using the online change request form on OGC web site: http://ogc.standardstracker.org/

Copyright notice

Note

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. The Open Geospatial Consortium shall not be held responsible for identifying any or all such patent rights.

Recipients of this document are requested to submit, with their comments, notification of any relevant patent claims or other intellectual property rights of which they may be aware that might be infringed by any implementation of the standard set forth in this document, and to provide supporting documentation.

CONTENTS

I ABSTRACT

OGC API Standards define modular API building blocks to spatially enable Web APIs in a consistent way. The OpenAPI specification is used to define the API building blocks.

This Standard ("Part 3 — Publish/Subscribe") defines how publish/subscribe protocols are used to asynchronously exchange CS API resources and their respective metadata and defined lifecycle events. It builds on the resource model and HTTP interface defined in OGC API — Connected Systems — Part 1 and on the dynamic data resource types defined in OGC API — Connected Systems — Part 2. It also extends the message and payload framework defined in OGC API — Pub/Sub by specifying the resource events, resource data messages, and protocol bindings specific to Connected Systems.

Part 3 provides a window into CS API and improves discoverability of resources as they become available for devices and systems already using popular pub/sub frameworks. It is intended to serve as a supplement to the REST API defined in Parts 1 & 2 while being robust enough that clients need not implement all both HTTP and Pub/Sub aspects of the standard.

Two categories of pub/sub messages are defined. **Resource events** are CloudEvents-formatted notifications that signal the creation, update, or deletion of any CS API resource; they carry a reference (and optionally a short summary) of the affected resource. **Resource data messages** carry full resource representations using the native CS API encodings (JSON, SWE Common JSON, SWE Common Text, SWE Common Binary) and are intended for low-latency streaming of CS API resources, as well as for the registration of new resources from publishing clients.

Protocol bindings define how these messages are exchanged over specific pub/sub transports. This Standard defines bindings for MQTT (v3 and v5), including topic/channel structure, subscribe and publish operations, optional content negotiation, and optional flow control. Additional bindings (e.g., AMQP, Kafka, NATS, Server-Sent Events(SSE)) may be defined in extensions or future revisions.

II KEYWORDS

The following are keywords to be used by search engines and document catalogues.

ogcdoc, OGC document, OpenAPI, AsyncAPI, REST, MQTT, CloudEvents, publish, subscribe, pub/sub, event, notification, API, system, smart system, connected system, IoT, sensorweb, ssn, sensor, actuator, observation, command

PREFACE

The OGC API — Connected Systems Standard is part of the suite of OGC API Standards.

To increase the brevity and readability of this Standard, many OGC document titles are shortened and/or abbreviated. Therefore, in the context of this document, the following phrases are defined.

- "this Standard" shall be interpreted as equivalent to "OGC API — Connected Systems — Part 3: Publish/Subscribe Standard."
- "CS API" or "CS API Standard" shall be interpreted as equivalent to "OGC API — Connected Systems Standard" (including all its parts).
- "OGC API — Features" shall be interpreted as equivalent to "OGC API — Features — Part 1: Core corrigendum."
- "OGC API — Common" shall be interpreted as equivalent to "OGC API — Common — Part 1: Core."
- "OGC API — Pub/Sub" shall be interpreted as equivalent to "OGC API — Publish-Subscribe Workflow — Part 1: Core."

IV SECURITY CONSIDERATIONS

Security considerations detailed in OGC API — Connected Systems — Part 1 and OGC API — Connected Systems — Part 2 also apply to this Standard. Additional security requirements specific to publish/subscribe interactions around the prevention of unauthorized publishing on server-controlled channels and the propagation of resource-level access control to event subscriptions are stated in the relevant requirements classes of this Standard.

V SUBMITTERS

All questions regarding this submission should be directed to the editor or the submitters listed in the following table:

NAME	AFFILIATION
Ian Patterson (Editor)	GeoRobotix Innovative Research
Alex Robin	GeoRobotix, Inc.
Christian Autermann	52°North Spatial Information Research GmbH
Chuck Heazel	Heazeltech
Glenn Laughlin	Pelagis Data Solutions
Mike Botts	Botts Innovative Research, Inc.
Patrick Cozzi	Cesium GS, Inc.
Sam Bolling	Riverside Research

Additional contributors to this Standard include the following:

NAME	AFFILIATION
Chris Tucker	GeoRobotix, Inc.

NAME	AFFILIATION
Qihua Li	GovTech Singapore
Rob Atkinson	Open Geospatial Consortium, Inc.
Simon Cox	Open Geospatial Consortium, Inc.
Jan Speckamp	52°North Spatial Information Research GmbH

1 SCOPE

1 SCOPE

This Standard defines protocol bindings and message payload formats for using publish/subscribe (pub/sub) protocols with the OGC API — Connected Systems (CS API) framework. It extends OGC API — Connected Systems — Part 1 (resource model and HTTP interface) and OGC API — Connected Systems — Part 2 (dynamic data resource types) by enabling clients and servers to exchange CS API resources, lifecycle notifications, and streaming data asynchronously over pub/sub transports.

Specifically, Part 3 of the CS API defines:

- Two main categories of pub/sub message payloads: **Resource Events** (CloudEvents-formatted lifecycle notifications) and **Resource Data Messages** (full resource representations carried in their native CS API encodings). A subtype of Resource Events **Batch Resource Events** (aggregated lifecycle notifications for high-volume cases), exists for cases where implementers require fewer message publications due to highly frequent lifecycle updates.
- Protocol bindings that specify how these message categories are exchanged over particular pub/sub transports. Bindings for MQTT (v3.1.1 and v5) are defined in this Standard. Additional bindings may be defined in extensions or in future revisions.
- An optional content-negotiation mechanism allowing a server to expose the same resource in multiple encoding formats over parallel pub/sub channels.

This Standard does not redefine resource models or HTTP operations defined in OGC API — Connected Systems — Part 1 or OGC API — Connected Systems — Part 2. It builds on the message-payload framework defined in OGC API — Pub/Sub — Part 1: Core, extending it for the Connected Systems domain.

OGC API — Connected Systems — Part 1 defines the feature resources that pub/sub channels in this Part exchange notifications and data about. OGC API — Connected Systems — Part 2 defines the dynamic data resource types (`Observation`, `Command`, `Command Status`, `Command Result`, `System Event`) that are most commonly streamed using the message categories defined here.

2 CONFORMANCE

2 CONFORMANCE

This Standard was written to be compliant with the OGC Specification Model – A Standard for Modular Specification (OGC 08-131r3). Extensions of this Standard shall themselves be conformant to the OGC Specification Model.

This Standard defines the following requirements classes.

- Clause 8, Requirements Class “Resource Events” defines requirements for CloudEvents-formatted notifications of CS API resource lifecycle operations (create, update, delete).
- Clause 9, Requirements Class “Batch Resource Events” defines requirements for CloudEvents-formatted aggregated lifecycle notifications, intended for high-volume cases such as observation streams.
- Clause 10, Requirements Class “Resource Data Messages” defines requirements for pub/sub messages carrying full CS API resource representations in their native encodings.
- Clause 10.4, Content Negotiation (Optional) defines an optional content-negotiation mechanism for Resource Data Messages, including a normalization rule for deriving channel-safe format-name suffixes from MIME types.
- [clause-protocol-mqtt], [clause-protocol-mqtt] defines requirements for using MQTT (v3.1.1 and v5) as a pub/sub transport for the message categories defined in this Standard.

The standardization target for these requirements classes is an implementation of the Web API.

There is no *Core* requirements class. An implementation target is expected to implement at least one message-category (Resource Events, Batch Resource Events, or Resource Data Messages) and at least one protocol-binding.

The conformance classes corresponding to these requirements classes are presented in Annex A (normative). Conformance with this Standard shall be checked using all the relevant tests specified in Annex A. The framework, concepts, and methodology for testing, and the criteria to be achieved to claim conformance are specified in the OGC Compliance Testing Policies and Procedures and the OGC Compliance Testing website.

3 NORMATIVE REFERENCES

3 NORMATIVE REFERENCES

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

Policy SWG, Open Geospatial Consortium: OGC 08-131r3, *The Specification Model — Standard for Modular specifications*. Open Geospatial Consortium (2009). https://docs.ogc.org/pol/08-131r3/08-131r3/pdf.

Alexandre Robin, Open Geospatial Consortium: OGC 23-001, *OGC API — Connected Systems — Part 1: Feature Resources*. Open Geospatial Consortium (2025). http://www.opengis.net/doc/IS/ogcapi-connectedsystems-1/1.0.

Alexandre Robin, Open Geospatial Consortium: OGC 23-002, *OGC API — Connected Systems — Part 2: Dynamic Data*. Open Geospatial Consortium (2025). http://www.opengis.net/doc/IS/ogcapi-connectedsystems-2/1.0.

OGC API — Publish-Subscribe Workflow — Part 1: Core, version 1.0 (DRAFT). Open Geospatial Consortium. https://docs.ogc.org/DRAFTS/25-030.html

Charles Heazel, Open Geospatial Consortium: OGC 19-072, *OGC API — Common — Part 1: Core*. Open Geospatial Consortium (2023). http://www.opengis.net/doc/is/ogcapi-common-1/1.0.0.

Clemens Portele, Panagiotis (Peter) A. Vretanos, Charles Heazel, Open Geospatial Consortium: OGC 17-069r4, *OGC API — Features — Part 1: Core corrigendum*. Open Geospatial Consortium (2022). http://www.opengis.net/doc/IS/ogcapi-features-1/1.0.1.

CloudEvents Specification, version 1.0.2. Cloud Native Computing Foundation. https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/spec.md

JSON Event Format for CloudEvents, version 1.0.2. Cloud Native Computing Foundation. https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/formats/json-format.md

Klyne G, Newman C, Internet Engineering Task Force: IETF RFC 3339, *Date and Time on the Internet: Timestamps*. RFC Publisher (2002). https://www.rfc-editor.org/info/rfc3339.

ISO: ISO 8601:2019, *Date and time — Representations for information interchange — Part 1: Basic rules. International Organization for Standardization, Geneva (2019).*. ISO (2019).

ISO: ISO 8601:2019, *Date and time — Representations for information interchange — Part 2: Extensions. International Organization for Standardization, Geneva (2019).*. ISO (2019).

Internet Engineering Task Force (committee): IETF RFC 8259, *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC Publisher (2017). https://www.rfc-editor.org/info/rfc8259.

Nottingham M: IETF RFC 8288, *Web Linking*. RFC Publisher (2017). https://www.rfc-editor.org/info/rfc8288.

Berners-Lee T, Fielding R, Masinter L: IETF RFC 3986, *Uniform Resource Identifier (URI): Generic Syntax*. RFC Publisher (2005). https://www.rfc-editor.org/info/rfc3986.

MQTT Version 5.0, 07 March 2019, OASIS Standard. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

MQTT Version 3.1.1 Plus Errata 01, 10 December 2015, OASIS Standard. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

4 TERMS AND DEFINITIONS

4 TERMS AND DEFINITIONS

This document uses the terms defined in OGC Policy Directive 49, which is based on the ISO/IEC Directives, Part 2, Rules for the structure and drafting of International Standards. In particular, the word "shall" (not "must") is the verb form used to indicate a requirement to be strictly followed to conform to this document and OGC documents do not use the equivalent phrases in the ISO/IEC Directives, Part 2.

This document also uses terms defined in the OGC Standard for Modular specifications (OGC 08-131r3), also known as the 'ModSpec'. The definitions of terms such as standard, specification, requirement, and conformance test are provided in the ModSpec.

For the purposes of this document, the following additional terms and definitions apply.

All terms defined in OGC API — Connected Systems — Part 1, OGC API — Connected Systems — Part 2, and OGC API — Pub/Sub — Part 1: Core also apply.

4.1. Channel

An addressable path or topic through which messages of a particular kind are published and to which clients can subscribe. The exact syntax of a channel identifier depends on the protocol binding (for example, an MQTT topic).

4.2. CloudEvents

A vendor-neutral specification for describing event data in a common way. CloudEvents are envelopes carrying context attributes (such as `id`, `type`, `source`, `subject`, `time`) and an optional payload (`data`).

[**SOURCE:** CloudEvents v1.0.2]

4.3. Resource Event

A CloudEvents-formatted message published by a CS API server to signal a lifecycle operation (create, update, or delete) on a single CS API resource.

4.4. Batch Resource Event

A CloudEvents-formatted message published by a CS API server to summarize a number of resource transactions of a single type that occurred on a single nested resource collection during a defined time window.

4.5. Resource Data Message

A pub/sub message whose payload is a complete representation of a single CS API resource, encoded in one of the server's supported native CS API encodings. Resource Data Messages do not use a CloudEvents envelope.

4.6. Publisher

The party (server or client, depending on the binding and channel) responsible for sending messages on a pub/sub channel.

4.7. Subscriber

A party that registers interest in receiving messages from one or more pub/sub channels.

4.8. Protocol Binding

A definition of how the abstract pub/sub message categories defined in this Standard map onto the concrete operations of a specific pub/sub transport protocol (for example, MQTT).

4.9. **Format Name**

A short, channel-safe identifier derived from a resource encoding's MIME type, used to distinguish parallel pub/sub channels carrying the same resource in different encodings. The normalization rule is defined in /req/resource-data-messages/content-negotiation/format-name.

6 CONVENTIONS

6 CONVENTIONS

This section provides details and examples for any conventions used in the document.

6.1. Identifiers

The normative provisions in this Standard are denoted by the URI

`http://www.opengis.net/spec/ogcapi-connectedsystems-3/1.0`

All requirements and conformance tests that appear in this document are denoted by partial URIs which are relative to this base.

6.2. Abbreviated terms

In this document the following abbreviations and acronyms are used or introduced:

- API: Application Programming Interface
- AsyncAPI: Specification for asynchronous APIs
- CS API: OGC API — Connected Systems
- HTTP: Hypertext Transfer Protocol
- IETF: Internet Engineering Task Force
- IoT: Internet of Things
- JSON: JavaScript Object Notation
- MIME: Multipurpose Internet Mail Extensions
- MQTT: Message Queuing Telemetry Transport
- OASIS: Organization for the Advancement of Structured Information Standards
- OGC: Open Geospatial Consortium
- Pub/Sub: Publish/Subscribe
- REST: Representational State Transfer

- RFC: Request for Comments
- SSE: Server-Sent Events
- SWE: Sensor Web Enablement
- URI: Uniform Resource Identifier
- URL: Uniform Resource Locator
- UUID: Universally Unique IDentifier

7 OVERVIEW

7 OVERVIEW

7.1. General

The OGC API — Connected Systems (CS API) defines a unified set of resource types and HTTP operations for interacting with sensors, actuators, platforms, and the dynamic data they produce. OGC API — Connected Systems — Part 1 and OGC API — Connected Systems — Part 2 define those resource types and the synchronous HTTP interface.

In many Connected Systems use cases, however, the request/response pattern of HTTP is not the most efficient way to exchange data. Sensors produce observations continuously; actuators must accept commands with low latency; subscribers want to know about new resources as soon as they appear, without polling. For these cases, an asynchronous publish/subscribe interaction model is more appropriate.

This Part of the CS API specifies how pub/sub protocols are used alongside the HTTP API to deliver the same resources and notifications about them asynchronously. The HTTP interface remains the authoritative way to retrieve current state; the pub/sub interface complements it with low-latency delivery and bidirectional streaming.

7.2. Message Categories

This Standard defines two primary categories of pub/sub messages and an extended secondary class, each addressing a different use case:

- **Resource Events** (Clause 8) are CloudEvents-formatted notifications of CS API resource lifecycle operations. They are emitted by the server when a resource is created, updated, or deleted. The notification carries a reference to the affected resource (and optionally a short summary) but not the resource itself.
- **Batch Resource Events** (Clause 9) CloudEvents-formatted extensions of Resource Events. They summarize a number of resource transactions of the same type that occurred on a single nested resource collection during a defined time window. They are intended primarily for high-frequency cases (for example, observation streams) where one event per transaction would be too verbose.
- **Resource Data Messages** (Clause 10) carry complete resource representations in the server's native CS API encodings (JSON, SWE Common JSON, SWE Common Text, SWE Common Binary, etc.). They are designed for low-latency streaming of observations, commands, command status, command results, and system events, as well as

for “registration” use cases where a publishing client introduces new systems, datastreams, or controlstreams to the server.

The three categories are designed to be used together though implementations are only required to implement one. A typical deployment exposes Resource Events for lifecycle notifications across resource types, Batch Resource Events for the highest-frequency nested collections, and Resource Data Messages for the dynamic data streams themselves.

7.3. Protocol Bindings

The message categories above are protocol-agnostic. To exchange these messages over a specific pub/sub transport, this Standard defines **protocol bindings**: requirements classes that specify how the abstract message categories map onto the concrete operations (subscribe, publish, topic structure, content negotiation, flow control) of each pub/sub transport.

This Standard defines bindings for:

- **MQTT** (v3.1.1 and v5) — broker-based pub/sub commonly used in IoT ([clause-protocol-mqtt]).

Additional bindings (for example, AMQP, Apache Kafka, Server-Sent Events) may be defined by future revisions or by extensions to this Standard.

7.4. Relationship to OGC API – Pub/Sub

This Standard extends OGC API — Pub/Sub — Part 1: Core by profiling its message-payload framework for the Connected Systems domain. The Resource Events and Batch Resource Events requirements classes both inherit from the OGC API – Pub/Sub `message-payload-cloudevents-json` requirements class.

Resource Data Messages do not inherit from any OGC API – Pub/Sub class, because the native CS API encodings (defined in OGC API — Connected Systems — Part 1 and OGC API — Connected Systems — Part 2) are not among the payload formats covered by OGC API – Pub/Sub Part 1.

8

REQUIREMENTS CLASS "RESOURCE EVENTS"

8 REQUIREMENTS CLASS "RESOURCE EVENTS"

REQUIREMENTS CLASS 1	
IDENTIFIER	`/req/resource-events`
TARGET TYPE	Web API
PREREQUISITE	https://www.opengis.net/spec/ogcapi-pubsub-1/1.0/req/message-payload-cloudevents-json
NORMATIVE STATEMENTS	Requirement 1: `/req/resource-events/cloudevents-format` Requirement 2: `/req/resource-events/event-properties` Requirement 3: `/req/resource-events/event-types` Requirement 4: `/req/resource-events/publisher-restriction` Requirement 5: `/req/resource-events/all-resource-types` Requirement 6: `/req/resource-events/event-discoverability`

8.1. Overview

Resource events are a mechanism by which a CS API client can be notified of certain resource operations:

- Resource creation (e.g. as initiated by an HTTP POST)
- Resource update (e.g. as initiated by an HTTP PUT or PATCH)
- Resource deletion (e.g. as initiated by an HTTP DELETE)

Listening for resource events can be done on a single resource or a valid collection of resources. The related topic will closely resemble a valid REST endpoint from parts 1 & 2.

Resource Events are encoded as CloudEvents v1.0 messages using the JSON Event Format.

8.2. Message Format

The following table specifies the constraints on each CloudEvents context attribute for a CS API Resource Event.

Table 1 — CloudEvents Context Attributes for Resource Events

ATTRIBUTE	VALUE / CONSTRAINT
`specversion`	Fixed value `1.0`.
`type`	A string identifying the affected resource type and the operation that took place. See Table 2.
`source`	The CS API root URL of the server publishing the event.
`subject`	The canonical URL of the CS API resource the event applies to.
`id`	A non-empty, producer-defined string that, together with `source`, uniquely identifies each distinct event. Producer-maintained counters and UUIDs are valid examples.
`parentId`	*(extension)* The canonical URL of the parent resource, when one exists (e.g. the parent `System` when a subsystem is created). Recommended whenever applicable.
`time`	The timestamp of the resource operation. Ideally the moment the resource becomes visible to API clients; alternatively, the time the originating request was received or processed by the server.
`datacontenttype`	Required only when `data` is included. SHALL be `application/json`.
`data`	*Optional.* When present, a JSON object summarizing the affected resource. Contains at least the resource's `name` and `description` properties when those are defined on the resource. Additional properties MAY be included.

Table 2 — Resource Event Types

EVENT TYPE	DESCRIPTION
`org.ogc.api.consys.<resource>.create`	A resource of the given type was created.
`org.ogc.api.consys.<resource>.update`	A resource of the given type was replaced or updated.
`org.ogc.api.consys.<resource>.delete`	A resource of the given type was deleted.

The `<resource>` token in the table above is replaced by the resource type identifier (for example `system`, `subsystem`, `datastream`, `observation`, `controlstream`, `command`, `commandstatus`,

`commandresult`, `systemevent`, `deployment`, `procedure`, `property`, `samplingfeature`). The complete mapping of resource types to event-type tokens is given in TODO (Annex).

Example 1 — Resource Event for a `DataStream` creation, with no `data` member: This is a minimal Resource Event notifying subscribers that a new `DataStream` resource has been created. The event carries only the canonical URL of the affected resource in the `subject` attribute. Subscribers may fetch the resource representation via HTTP directly, given the canonical url.

```
{
  "id": "6e1c7f9f-dd6c-48d9-bbc4-aef0625f1fb8",
  "specversion": "1.0",
  "type": "org.ogc.api.consys.datastream.create",
  "source": "https://example.org/csapi",
  "subject": "https://example.org/csapi/datastreams/458",
  "time": "2025-08-27T12:36:33Z"
}
```

Example 2 — Resource Event for a `System` creation, with a `data` summary: This Resource Event signals the creation of a new `System` resource and includes an optional `data` member with a short summary of the resource.

```
{
  "id": "8b7a6c5d-4e3f-4210-9876-543210fedcba",
  "specversion": "1.0",
  "type": "org.ogc.api.consys.system.create",
  "source": "https://example.org/csapi",
  "subject": "https://example.org/csapi/systems/87",
  "time": "2025-10-07T10:23:30Z",
  "datacontenttype": "application/json",
  "data": {
    "name": "Sensor 01",
    "description": "Air temperature sensor on my window"
  }
}
```

8.3. Requirements

REQUIREMENT 1	
IDENTIFIER	`/req/resource-events/cloudevents-format`
INCLUDED IN	Requirements class 1: `/req/resource-events`
A	The payload of a Resource Event SHALL be a CloudEvents v1.0 message encoded using the Cloud Events JSON Event Format 1.0.2.

REQUIREMENT 2	
IDENTIFIER	`/req/resource-events/event-properties`
INCLUDED IN	Requirements class 1: `/req/resource-events`
A	A Resource Event SHALL set the CloudEvents context attributes as defined in Table 1.
B	The `id` attribute SHALL be a non-empty string. The combination of the `source` and `id` attributes SHALL uniquely identify each distinct Resource Event. A retransmission of the same Resource Event MAY reuse the same combination.
C	The `source` attribute SHALL be set to the CS API root URL of the server publishing the event.
D	The `subject` attribute SHALL be set to the canonical URL of the CS API resource the event applies to.
E	When the resource the event applies to has a parent resource, the `parentId` extension attribute SHALL be set to the canonical URL of that parent resource.
F	When the event payload includes a `data` member, the `datacontenttype` attribute SHALL be set to `application/json`.
G	When present, the `data` member SHALL be a JSON object containing at least the `name` and `description` properties of the resource, where those properties are defined on the resource.

REQUIREMENT 3	
IDENTIFIER	`/req/resource-events/event-types`
INCLUDED IN	Requirements class 1: `/req/resource-events`
A	The `type` attribute of a Resource Event SHALL be one of the values listed in Table 2.
B	For every CS API resource type exposed by the server, the server SHALL publish events with the `.create`, `.update`, and `.delete` type suffixes when the corresponding resource operations occur.

REQUIREMENT 4	
IDENTIFIER	`/req/resource-events/publisher-restriction`
INCLUDED IN	Requirements class 1: `/req/resource-events`
A	Only the CS API server hosting the resources SHALL publish Resource Events for those resources.

REQUIREMENT 4	
B	The server SHALL reject any attempt by a client to publish a message on a channel reserved for Resource Events.

REQUIREMENT 5	
IDENTIFIER	`/req/resource-events/all-resource-types`
INCLUDED IN	Requirements class 1: `/req/resource-events`
A	A server implementing this requirements class SHOULD publish Resource Events for every resource type that it exposes through any other requirements class of this Standard.
B	A server MAY restrict the set of resource types and/or operations for which it publishes Resource Events. When restricted, the server SHALL declare the supported set via the mechanism defined in /req/resource-events/event-discoverability.

REQUIREMENT 6	
IDENTIFIER	`/req/resource-events/event-discoverability`
INCLUDED IN	Requirements class 1: `/req/resource-events`
A	A server implementing this requirements class SHALL make the set of event types it publishes discoverable to clients.
B	When the server restricts the set of resource types and/or operations for which it publishes events (see /req/resource-events/all-resource-types), the discoverability response SHALL enumerate exactly the supported `<resource>.<operation>` tokens.
STATEMENT	TODO The mechanism by which the server advertises its supported event types is not yet specified. Candidates under consideration include: • An AsyncAPI document published at a well-known endpoint of the API, with one channel per supported event type. • A conformance-class declaration enumerating the supported `<resource>.<operation>` tokens. • Link relations from the API root or a dedicated `/events` endpoint pointing at per-resource-type event channels. This requirement will be tightened in a future revision of this Standard once the chosen mechanism is finalized.

9

REQUIREMENTS CLASS "BATCH RESOURCE EVENTS"

9 REQUIREMENTS CLASS "BATCH RESOURCE EVENTS"

REQUIREMENTS CLASS 2	
IDENTIFIER	`/req/batch-resource-events`
TARGET TYPE	Web API
PREREQUISITE	https://www.opengis.net/spec/ogcapi-pubsub-1/1.0/req/message-payload-cloudevents-json
NORMATIVE STATEMENTS	Requirement 7: `/req/batch-resource-events/cloudevents-format` Requirement 8: `/req/batch-resource-events/event-properties` Requirement 9: `/req/batch-resource-events/event-types` Requirement 10: `/req/batch-resource-events/publisher-restriction` Requirement 11: `/req/batch-resource-events/batch-data-shape` Requirement 12: `/req/batch-resource-events/allowed-collections`

9.1. Overview

Batch Resource Events are an alternative to per-resource Resource Events, intended for cases where publishing one event per resource transaction is impractical — for example, high-bandwidth datastreams and control streams where the per-resource event volume would overwhelm subscribers.

A Batch Resource Event is a CloudEvents v1.0 message that summarizes a number of resource transactions of a single type (create, update, or delete) that occurred on a single nested resource collection during a defined time window. It does not carry the individual resources or their identifiers; only their count, the event type, and the time window.

Implementing this requirements class is **optional**, even for servers that implement Resource Events. A server MAY implement either class, both, or neither.

Batch Resource Events are **not** intended for low-frequency resource collections such as the canonical collections of systems, deployments, procedures, properties, or the per-system collections of subsystems, datastreams, controlstreams, sampling features, or deployments. For those collections, per-resource Resource Events are sufficient and avoid the additional complexity of batch aggregation. The specific resource collections for which Batch Resource Events MAY be published are enumerated in /req/batch-resource-events/allowed-collections.

9.2. Message Format

A Batch Resource Event uses the same CloudEvents v1.0 JSON envelope as a regular Resource Event (see Table 1) with a few key differences:

- The `subject` attribute identifies a **nested resource collection** (resources endpoint) that the batch summarizes, rather than an individual resource. The collection is nested under exactly one parent resource (e.g. observations under a single datastream). Top-level resource collections and collections that span multiple parent resources are not allowed; the permitted collections are enumerated in /req/batch-resource-events/allowed-collections.
- The `data` member is **required** and carries summary statistics for the batched operations.

Table 3 — Batch Resource Event property differences from Resource Events

ATTRIBUTE	VALUE / CONSTRAINT
`subject`	The canonical URL of the nested resource collection (resources endpoint) summarized by the batch. SHALL be one of the collections enumerated in /req/batch-resource-events/allowed-collections.
`datacontenttype`	Required. SHALL be `application/json`.
`data`	Required. JSON object carrying batch statistics; see Table 4.

All other CloudEvents context attributes (`specversion`, `type`, `source`, `id`, `parentId`, `time`) follow the same constraints as for regular Resource Events.

Table 4 — Batch Resource Event `data` properties

PROPERTY	TYPE	DESCRIPTION
`timerange`	Array of two RFC 3339 date-time strings	The window during which the batched resource operations became visible to API clients. The two values SHALL be in chronological order.
`count`	Non-negative integer	The number of resource operations of the event type identified by `type` that occurred on the resource collection identified by `subject` during the `timerange`.

Additional properties MAY be included in `data` (for example, per-observed-property statistics or latency metrics). Consumers SHALL ignore any properties they do not recognize.

The `type` vocabulary is the same as for regular Resource Events; see Table 2.

Example — Batch Resource Event summarizing one minute of observation creations: This Batch Resource Event reports that 1254 `Observation` resources were created under datastream 132 during a one-minute window. Individual observations are not enumerated; subscribers wishing to retrieve them can use the HTTP API or subscribe to per-observation Resource Events.

```
{
  "id": "f1e2d3c4-b5a6-4798-8765-432109876543",
  "specversion": "1.0",
  "type": "org.ogc.api.consys.observation.create",
  "source": "https://example.org/csapi",
  "subject": "https://example.org/csapi/datastreams/132/observations",
  "time": "2025-10-07T10:01:00.025Z",
  "datacontenttype": "application/json",
  "data": {
    "timerange": ["2025-10-07T10:00:00Z", "2025-10-07T10:01:00Z"],
    "count": 1254
  }
}
```

9.3. Requirements

REQUIREMENT 7	
IDENTIFIER	`/req/batch-resource-events/cloudevents-format`
INCLUDED IN	Requirements class 2: `/req/batch-resource-events`
A	The payload of a Batch Resource Event SHALL be a CloudEvents v1.0 message encoded using the CloudEvents JSON Event Format 1.0.2.

REQUIREMENT 8	
IDENTIFIER	`/req/batch-resource-events/event-properties`
INCLUDED IN	Requirements class 2: `/req/batch-resource-events`
A	A Batch Resource Event SHALL set the CloudEvents context attributes as defined in Table 3.
B	The `id` attribute SHALL be a non-empty string. The combination of the `source` and `id` attributes SHALL uniquely identify each distinct Batch Resource Event. A retransmission of the same Batch Resource Event MAY reuse the same combination.

REQUIREMENT 8

C	The `source` attribute SHALL be set to the CS API root URL of the server publishing the event.
D	The `subject` attribute SHALL be set to the canonical URL of the CS API resource collection (resources endpoint) that the batch summarizes. The collection SHALL be one of those enumerated in /req/batch-resource-events/allowed-collections.
E	The `parentId` extension attribute SHOULD be set to the canonical URL of the parent resource under which the summarized collection is nested.
F	The `data` member SHALL be present.
G	The `datacontenttype` attribute SHALL be set to `application/json`.

REQUIREMENT 9

IDENTIFIER	`/req/batch-resource-events/event-types`
INCLUDED IN	Requirements class 2: `/req/batch-resource-events`
A	The `type` attribute of a Batch Resource Event SHALL be one of the values listed in Table 2.
B	A single Batch Resource Event SHALL summarize resource transactions of exactly one event type (e.g. only `.create`, only `.update`, or only `.delete`) for resources of exactly one type.

REQUIREMENT 10

IDENTIFIER	`/req/batch-resource-events/publisher-restriction`
INCLUDED IN	Requirements class 2: `/req/batch-resource-events`
A	Only the CS API server hosting the resources SHALL publish Batch Resource Events for those resources.
B	The server SHALL reject any attempt by a client to publish a message on a channel reserved for Batch Resource Events.

REQUIREMENT 11

IDENTIFIER	`/req/batch-resource-events/batch-data-shape`

REQUIREMENT 11	
INCLUDED IN	Requirements class 2: `/req/batch-resource-events`
A	The `data` member of a Batch Resource Event SHALL be a JSON object containing at least the `timerange` and `count` properties defined below.
B	The `timerange` property SHALL be a JSON array of exactly two RFC 3339 date-time strings, in chronological order, representing the window during which the batched resource operations became visible to API clients.
C	The `count` property SHALL be a non-negative integer representing the number of resource operations of the event type identified by `type` that occurred on the resource collection identified by `subject` during the `timerange`.
D	The `data` member MAY include additional properties beyond `timerange` and `count`. Consumers SHALL ignore any properties they do not recognize.

REQUIREMENT 12	
IDENTIFIER	`/req/batch-resource-events/allowed-collections`
INCLUDED IN	Requirements class 2: `/req/batch-resource-events`
A	A server SHALL publish Batch Resource Events only on resource collections that are nested under exactly one parent resource. Batch Resource Events SHALL NOT be published for top-level resource collections, nor for collections that span multiple parent resources.
B	A server implementing this requirements class SHALL only publish Batch Resource Events whose `topic` is the canonical URL of one of the following nested resource collections: • `{api_root}/datastreams/{dsId}/observations` • `{api_root}/controlstreams/{csId}/commands` • `{api_root}/controlstreams/{csId}/commands/{cmdId}/status` • `{api_root}/controlstreams/{csId}/commands/{cmdId}/result` • `{api_root}/systems/{sysId}/systemEvents`

10

REQUIREMENTS CLASS "RESOURCE DATA MESSAGES"

10 REQUIREMENTS CLASS "RESOURCE DATA MESSAGES"

REQUIREMENTS CLASS 3	
IDENTIFIER	`/req/resource-data-messages`
TARGET TYPE	Web API
NORMATIVE STATEMENTS	Requirement 13: `/req/resource-data-messages/payload-format` Requirement 14: `/req/resource-data-messages/no-deletes` Requirement 15: `/req/resource-data-messages/allowed-resource-types`

10.1. Overview

Resource Data Messages allow CS API resources to be exchanged through publish/subscribe channels as their native representations rather than as Resource Events. They are designed for low-latency, high-volume streaming of dynamic data — observations, commands, command status updates, command results, and system events — as well as for "registration" use cases where a publishing client introduces new systems, datastreams, or controlstreams to the server.

A Resource Data Message carries the resource itself, not a notification about it. Subscribers receive the full resource representation; consumers do not need to make a follow-up request to retrieve the resource by URL.

Resource Data Messages are restricted in two important ways:

- Only specific CS API resource types are eligible (see Table 5). The Resource Events class remains the appropriate mechanism for lifecycle notifications of other resource types (deployments, procedures, properties, sampling features, etc.).
- Resource deletions are **never** signaled by Resource Data Messages. Deletions are conveyed only by Resource Events.

Implementing this requirements class is optional. A server MAY implement Resource Data Messages without implementing Resource Events, or vice versa, or both, or neither.

10.2. Message Format

A Resource Data Message carries a complete representation of a single CS API resource. Unlike Resource Events and Batch Resource Events, the message payload is **not** wrapped in a CloudEvents envelope — it is the resource representation itself, encoded in one of the formats supported by the server.

Table 5 — CS API resource types eligible for Resource Data Messages

RESOURCE TYPE	DEFINED IN	TYPICAL USE CASE
`Observation`	OGC API — Connected Systems — Part 2	Streaming observations into or out of the server.
`Command`	OGC API — Connected Systems — Part 2	Sending commands to actuators or controllable systems.
`Command Status`	OGC API — Connected Systems — Part 2	Streaming command status transitions.
`Command Result`	OGC API — Connected Systems — Part 2	Streaming command results.
`System Event`	OGC API — Connected Systems — Part 2	Streaming system-emitted events.
`System`	OGC API — Connected Systems — Part 1	Registering a new system from the client side.
`Datastream`	OGC API — Connected Systems — Part 2	Registering a new datastream associated with a system.
`Controlstream`	OGC API — Connected Systems — Part 2	Registering a new controlstream associated with a system.

Resource Data Messages support both server-to-client and client-to-server message flows. The publish authority for each resource type and channel is defined by the protocol binding (for example, the MQTT Publish requirements class); the message-format class defined here is silent on who may publish.

The encoding used for a given Resource Data Message is one of the formats supported by the server for the resource type being carried, as defined by the encoding requirements classes of OGC API — Connected Systems — Part 1, OGC API — Connected Systems — Part 2, or future Parts. The mechanism by which a client requests a specific encoding (when more than one is supported by the server) is a protocol binding concern; this Standard does not define a general content-negotiation mechanism for Resource Data Messages.

10.3. Requirements

REQUIREMENT 13	
IDENTIFIER	`/req/resource-data-messages/payload-format`
INCLUDED IN	Requirements class 3: `/req/resource-data-messages`
A	The payload of a Resource Data Message SHALL be a complete representation of a single CS API resource, encoded in one of the resource encoding formats defined by the requirements classes of OGC API — Connected Systems — Part 1, OGC API — Connected Systems — Part 2, or this Standard.
B	The encoding format used for a given Resource Data Message SHALL be one of the formats that the server has declared as supported for the resource type being carried.

REQUIREMENT 14	
IDENTIFIER	`/req/resource-data-messages/no-deletes`
INCLUDED IN	Requirements class 3: `/req/resource-data-messages`
A	Resource Data Messages SHALL NOT be published to signal a resource deletion.
B	Deletion of a CS API resource SHALL be conveyed only by Resource Events.

REQUIREMENT 15	
IDENTIFIER	`/req/resource-data-messages/allowed-resource-types`
INCLUDED IN	Requirements class 3: `/req/resource-data-messages`
A	A server implementing this requirements class SHALL only publish and accept, Resource Data Messages whose payload is a representation of one of the following CS API resource types: • `Observation` (defined in OGC API — Connected Systems — Part 2) • `Command` (defined in OGC API — Connected Systems — Part 2) • `Command Status` (defined in OGC API — Connected Systems — Part 2) • `Command Result` (defined in OGC API — Connected Systems — Part 2) • `System Event` (defined in OGC API — Connected Systems — Part 2)

REQUIREMENT 15
• `System` (defined in OGC API — Connected Systems — Part 1) • `Datastream` (defined in OGC API — Connected Systems — Part 2) • `Controlstream` (defined in OGC API — Connected Systems — Part 2)

10.4. Content Negotiation (Optional)

REQUIREMENTS CLASS 4	
IDENTIFIER	`/req/resource-data-messages/content-negotiation`
TARGET TYPE	Web API
PREREQUISITE	Requirements class 3: `/req/resource-data-messages`
NORMATIVE STATEMENT	Requirement 16: `/req/resource-data-messages/content-negotiation/format-name`

10.4.1. Overview

Content negotiation for Resource Data Messages is an **optional** extension to the Resource Data Messages requirements class. When a server supports more than one resource encoding format for the same resource type, it MAY expose each format on a separate pub/sub channel identified by a normalized format-name suffix.

The actual channel-naming syntax (for example, an MQTT subtopic appended after `:data`) is defined by each protocol binding. This requirements class defines only the rule for deriving the format-name suffix from a MIME type. The rule is constrained primarily by MQTT, where the `+` character is a reserved single-level wildcard in topic filters and cannot appear inside a literal topic segment.

Table 6 — Normalized format names for common CS API encodings

MIME TYPE	DEFINED IN	NORMALIZED FORMAT NAME
`application/json`	OGC API — Connected Systems — Part 1, OGC API — Connected Systems — Part 2	`json`
`application/geo+json`	OGC API — Connected Systems — Part 1	`geo-json`

MIME TYPE	DEFINED IN	NORMALIZED FORMAT NAME
`application/sml+json`	OGC API — Connected Systems — Part 1	`sml-json`
`application/om+json`	OGC API — Connected Systems — Part 2	`om-json`
`application/cmd+json`	OGC API — Connected Systems — Part 2	`cmd-json`
`application/swe+json`	OGC API — Connected Systems — Part 2	`swe-json`
`application/swe+text`	OGC API — Connected Systems — Part 2	`swe-text`
`application/swe+binary`	OGC API — Connected Systems — Part 2	`swe-binary`

NOTE: This table is informative. The normative rule for deriving format names is given by /req/resource-data-messages/content-negotiation/format-name; the table above illustrates the rule applied to the encoding MIME types defined by the current Parts of the CS API Standard. Future encoding MIME types defined by extensions or future Parts derive their format names by applying the same rule.

10.4.2. Requirements

REQUIREMENT 16	
IDENTIFIER	`/req/resource-data-messages/content-negotiation/format-name`
INCLUDED IN	Requirements class 4: `/req/resource-data-messages/content-negotiation`
A	The **format name** used to identify a specific resource encoding within a pub/sub channel SHALL be derived from the encoding's MIME type by applying the following normalization, in order: 1. Take the bare media type, discarding any parameters (everything from the first `;` character, if present, onward). 2. Remove the `application/` prefix, if present. 3. Replace every + character with a – character. 4. Lowercase all remaining characters.
B	The normalized format name SHALL be used wherever a format identifier is required by a protocol binding, including MQTT topic segments and equivalent URL path segments in other bindings. The substitution of + with – is required because + is a reserved single-level wildcard in MQTT topic filters.

A

ANNEX A (INFORMATIVE) REVISION HISTORY

ANNEX A (INFORMATIVE) REVISION HISTORY

DATE	RELEASE	EDITOR	PRIMARY CLAUSES MODIFIED	DESCRIPTION
2026-05	1.0 draft	Ian Patterson	All	Initial draft skeleton: Resource Events, Batch Resource Events, Resource Data Messages, Content Negotiation sub-class; front matter; bibliography.

BIBLIOGRAPHY

BIBLIOGRAPHY

[1] Tom Kralidis, Mark Burgoyne, Steve Olson, Shane Mill, Open Geospatial Consortium: OGC 23-013, *Discussion paper for Publish-Subscribe workflow in OGC APIs*. Open Geospatial Consortium (2023). http://www.opengis.net/doc/discussion-paper/ogcapi-pubsub/1.0.

[2] Katharina Schleidt, Ilkka Rinne, Open Geospatial Consortium: OGC 20-082r4, *Topic 20 — Observations, measurements and samples*. Open Geospatial Consortium (2023). http://www.opengis.net/doc/as/om/3.0.

[3] Tom Kralidis, Chris Little, Mark Burgoyne, Steve Olson, Shane Mill, Open Geospatial Consortium: OGC 23-057r1, *OGC API — Environmental Data Retrieval — Part 2: Publish-Subscribe Workflow*. Open Geospatial Consortium (2024). http://www.opengis.net/doc/IS/ogcapi-edr-2/1.0.

[4] Steve Liang, Tania Khalafbeigi, Hylke van der Schaaf, Open Geospatial Consortium: OGC 18-088, *OGC SensorThings API Part 1: Sensing Version 1.1*. Open Geospatial Consortium (2021). http://www.opengis.net/doc/is/sensorthings/1.1.0.

[5] Steve Liang, Tania Khalafbeigi, Open Geospatial Consortium: OGC 17-079r1, *OGC SensorThings API Part 2 – Tasking Core*. Open Geospatial Consortium (2019). http://www.opengis.net/doc/IS/sensorthings-part2-TaskingCore/1.0.0.

[6] *CloudEvents Subscriptions API*. Cloud Native Computing Foundation. https://github.com/cloudevents/spec/blob/main/subscriptions/spec.md

[7] *AsyncAPI Specification, version 2.6.0*. https://www.asyncapi.com/docs/reference/specification/v2.6.0